Test Coverage Report

Mentify Backend

Measurement date: 25 October 2026, 23:25 (local time)
Source: Local JaCoCo XML and Maven Surefire XML from the backend working tree
Scope: 14 business service modules in `services/pom.xml`; local Maven Surefire execution
Acceptance thresholds: 80% line coverage and 70% branch coverage (accepted by the project owner)

1. Executive Summary

Verification: The full backend command `clean verify -DskipITs=true` completed with exit code 0 for this measurement.

Metric	Covered	Missed	Total	Coverage	Target	Result
Lines	10,372	2,446	12,818	80.92%	80%	PASS
Branches	4,103	1,652	5,755	71.29%	70%	PASS

Surefire results: 1,156 tests, 0 failures, 0 errors, 1 skipped, across 197 suites. These totals cover only the 14 service modules listed below.

Calculation method: The overall figures are weighted. Covered counters are summed across all 14 service JaCoCo reports, then divided by the summed covered and missed counters. They are not the arithmetic mean of the service percentages.

2. Service and Branch Coverage Breakdown

In this report, "branch" means a JaCoCo decision branch in application code, not a Git branch.

Service	Lines covered / total	Line %	Branches covered / total	Branch %	Tests
ai-service	1,957 / 2,331	83.96%	924 / 1,230	75.12%	188
analytics-service	50 / 61	81.97%	11 / 18	61.11%	13
assignment-service	1,761 / 2,281	77.20%	607 / 916	66.27%	138
attendance-service	333 / 421	79.10%	115 / 152	75.66%	31
auth-service	243 / 322	75.47%	67 / 90	74.44%	24
calender-service	392 / 590	66.44%	255 / 398	64.07%	64
communication-service	641 / 702	91.31%	238 / 318	74.84%	102
course-service	1,832 / 2,090	87.66%	804 / 1,048	76.72%	182
entrollment-service	294 / 365	80.55%	103 / 130	79.23%	31
exam-service	635 / 814	78.01%	179 / 316	56.65%	117
material-service	894 / 1,023	87.39%	402 / 540	74.44%	101
notification-service	129 / 178	72.47%	42 / 72	58.33%	10
payment-service	466 / 583	79.93%	130 / 204	63.73%	80
user-service	745 / 1,057	70.48%	226 / 323	69.97%	75
TOTAL	**10,372 / 12,818**	**80.92%**	**4,103 / 5,755**	**71.29%**	**1,156**

A service percentage below the overall target does not change the weighted aggregate result. The table exposes those service-level gaps; the 80% / 70% acceptance stated above applies to the aggregate only.

Services below the aggregate targets (for follow-up):

- **Line coverage below 80%:** assignment-service (77.20%), attendance-service (79.10%), auth-service (75.47%), calender-service (66.44%), exam-service (78.01%), notification-service (72.47%), payment-service (79.93%), user-service (70.48%).
- **Branch coverage below 70%:** analytics-service (61.11%), assignment-service (66.27%), calender-service (64.07%), exam-service (56.65%), notification-service (58.33%), payment-service (63.73%), user-service (69.97%).

3. Additional Backend Modules and Broader Total

The root Maven reactor also includes `common-lib` and `cloud`. The two infrastructure modules that produced JaCoCo XML are listed below. `cloud/config-server` and `cloud/service-registry` produced no JaCoCo XML and therefore have no measured coverage in this report.

Module	Lines covered / total	Line %	Branches covered / total	Branch %
common-lib	148 / 437	33.87%	58 / 212	27.36%
cloud/api-gateway	53 / 87	60.92%	0 / 18	0.00%
All 16 reported modules	**10,573 / 13,342**	**79.25%**	**4,161 / 5,985**	**69.52%**

The 80% / 70% acceptance result above is the 14-service business-layer result. The broader 16-report total is disclosed separately and is not labelled as meeting that target, because its own percentages (79.25% lines, 69.52% branches) do not. Modules without JaCoCo XML are outside both denominators.

4. How to Reproduce the Measurement

Prerequisites: A JDK and Maven compatible with the backend `pom.xml`, and backend dependencies available to Maven.

Run from the backend repository root:

```
mvn -B --no-transfer-progress clean verify -DskipITs=true
```

- `clean` removes earlier build outputs.
- `verify` runs tests, creates HTML/XML JaCoCo reports, and applies the backend's configured JaCoCo checks.
- `-DskipITs=true` requests that Failsafe integration tests be skipped; tests named `IntegrationTest` that are included by Surefire can still run.
- A successful exit code is required before using the generated XML as final evidence.

Report locations:

- HTML report: `services/<service>/target/site/jacoco/index.html`
- Machine-readable report: `services/<service>/target/site/jacoco/jacoco.xml`
- Surefire files: `services/<service>/target/surefire-reports/TEST-*.xml`

Regenerating this report: To regenerate both the Markdown report and its PDF from those XML files, run from the test repository root:

```
python tools/build-test-coverage-report.py E:\ResumeProjects\Mentify-backend-testing
```

Replace the final argument with the path to your backend checkout. The script requires PyMuPDF (`fitz`) for PDF creation. The XML source is the authority for the values above.

5. Interpretation and Evidence Limits

- JaCoCo counts executed lines and decision branches in instrumented backend Java classes. This report does not measure frontend code or the separate test repository's Java code.
- The accepted aggregate includes only the 14 business services in `services/pom.xml`. The separate 16-report total adds `common-lib` and `cloud/api-gateway`. Neither represents changed-code coverage or a single combined cross-module JVM report.
- Unit and module test coverage is distinct from live API, external AI, performance and staging results. Those require their own environment-specific evidence.
- The backend Maven build also has an 80% line/branch gate for selected auth/security classes and a 90% per-class branch gate for two SDS grading classes. Passing the 80% / 70% aggregate here does not by itself prove those separate checks; a successful `verify` run does.
- This report is derived from local generated XML, not a named GitHub Actions artifact. For commit-specific submission evidence, retain the successful CI run ID, backend SHA, and matching JaCoCo artifacts from that run.

Timestamp range: Report files were generated between 2026-10-25 23:17 and 23:25 (local). All service XML files should come from the same clean verification run; if the range is unexpectedly wide, rerun `clean verify` before submission.